



Egypt-Japan University of Science and Technology
الجامعة المصرية اليابانية للعلوم و التكنولوجيا
エジプト日本科学技術大学

AURA Customer Ordering System

Documentation Package

D2 · D3 · D4

Repository: Customer-Ordering-System

Subsystem: Customer Ordering System

Stack: Python Flask · Firebase · SQLAlchemy / SQLite

Submitted by:

Name	University ID
Mariam Gamal Elsayed	120230012
Mona Mohamed	120230016
Nancy Ahmed	120230045
Kirolos Khairy Shawkey	120230237
Marisia Micheal	120230233

Contents

1	Requirements Report (D2)	3
1.1	Actor Classification	3
1.2	Functional Requirements	3
1.2.1	Authentication & RBAC	3
1.2.2	Referral & Onboarding	3
1.2.3	Catalog & Cart	3
1.2.4	Checkout & Orders	4
1.2.5	Admin	4
1.2.6	Delivery	4
1.2.7	Profile, Rewards & Notifications	4
1.3	Persona Edge Cases	5
1.4	Traceability Heatmap	6
1.4.1	Matrix 1: Authentication, RBAC, and Referral Mapping	6
1.4.2	Matrix 2: Catalog, Cart, and Checkout Mapping	7
1.4.3	Matrix 3: Orders, Admin, Delivery, and System Configurations	8
2	Design Specification (D3)	9
2.1	Gherkin Scenarios	9
2.1.1	Authentication	9
2.1.2	Cart & Checkout	9
2.1.3	Order Cancellation	9
2.1.4	Admin	10
2.1.5	Delivery	10
2.1.6	Voucher & Wishlist	10
2.2	QA Refinement	10
2.3	UML Diagrams (Mermaid)	11
2.4	API Contracts	16
2.4.1	Auth (/auth)	16
2.4.2	Customer (/customer)	16
2.4.3	Order & Profile Routes	17
2.4.4	Admin (/admin) — requires <code>admin</code> role	17
2.4.5	Delivery (/delivery) — requires <code>delivery</code> role	17
2.4.6	Dead Code / Not Registered	17
2.5	Information Hiding / Interfaces	18
3	Sprint Breakdown	19
4	Test Code	25
4.1	<code>conftest.py</code> — Shared Fixtures	25
4.2	<code>test_auth.py</code> — Authentication Tests	25
4.3	<code>test_cart.py</code> — Cart & Stock Tests	25
4.4	<code>test_checkout.py</code> — Checkout & Pricing Tests	25
4.5	<code>test_order.py</code> — Order Cancel & Tracking Tests	25
4.6	<code>test_delivery.py</code> — Delivery Pipeline Tests	25
4.7	<code>test_admin.py</code> — Admin Tests	25
4.8	<code>test_rewards.py</code> — Loyalty & Points Tests	25
4.9	<code>test_pages.py</code> — Static Pages Tests	25
4.10	<code>test_card_validation.py</code> — Card Validation Tests	26
5	Validation Report (D4)	27

5.1	Testing Pyramid	27
5.2	Verification vs Validation	27
5.3	TDP Flow	27
5.4	Edge Case Cage (Padlocks)	27
5.5	Test Report Template	27
6	Full Traceability Matrix	29
7	Playwright Mapping (Planned)	30
8	Repository Gaps Disclosure	31

1. Requirements Report (D2)

1.1 Actor Classification

Actor Type	Actor	Rationale (repository)
Primary	Customer (role: customer)	Registers via Firebase, browses menu, cart, checkout, tracks orders (<code>customer.py</code> , <code>profile.py</code> , <code>order.py</code>)
Supporting	Admin / Manager (role: admin)	<code>/admin/*</code> dashboard, orders, menu (<code>admin.py</code> , <code>@role_required("admin")</code>)
Supporting	Delivery staff (role: delivery)	<code>/delivery/*</code> pipeline (<code>delivery.py</code> , <code>@role_required("delivery")</code>)
Offstage	Firebase Identity Toolkit	Customer register/login (<code>auth.py</code>)
Offstage	Email verification channel	Firebase <code>sendOobCode VERIFY_EMAIL</code> on register

1.2 Functional Requirements

1.2.1 Authentication & RBAC

- **REQ-AUTH-01:** Customer registration requires username ≥ 3 chars, email with @ and domain dot, password ≥ 8 chars (`auth.py _validate_registration`).
- **REQ-AUTH-02:** Optional phone must match regex `^[1-9]\d{6,14}$` if provided.
- **REQ-AUTH-03:** Registration creates Firebase user when `FIREBASE_WEB_API_KEY` configured; otherwise HTTP 500 “Firebase not configured.”
- **REQ-AUTH-04:** Customer login requires Firebase `auth + emailVerified == true` or login rejected with verify-email message.
- **REQ-AUTH-05:** Staff login uses Werkzeug `check_password_hash` only when `user.uses_staff_authentication()`.
- **REQ-AUTH-06:** Post-login redirect: admin $\rightarrow$ `/admin/`, delivery $\rightarrow$ `/delivery/`, customer $\rightarrow$ `/customer/menu`.
- **REQ-RBAC-01:** Unauthorized role access returns HTTP 403.
- **REQ-RBAC-02:** Unauthenticated access redirects to `auth.login` (or JSON 401 for AJAX).

1.2.2 Referral & Onboarding

- **REQ-REF-01:** Optional referral code validated at register; invalid code $\rightarrow$ 400 JSON error.
- **REQ-REF-02:** On valid referral: Referral row, referrer voucher REFER25 (25%, min \$20, 60 days).
- **REQ-REF-03:** New customer receives welcome voucher WELCOME20.
- **REQ-REF-04:** On first DELIVERED order, referred user receives FRIEND15-`{id}` voucher.

1.2.3 Catalog & Cart

- **REQ-MENU-01:** Browse menu by category or all.
- **REQ-MENU-02:** Search menu.

- **REQ-MENU-03:** Product details with reviews and wishlist flag.
- **REQ-CART-01:** Session cart add/update/remove (`session['cart']`, `CartService`).
- **REQ-CART-02:** Add quantity cannot exceed `menu_item.available_stock`.
- **REQ-CART-03:** Public promos SAVE10, AURA20, SPRING15 with expiry and min order.

1.2.4 Checkout & Orders

- **REQ-CHK-01:** Checkout requires login.
- **REQ-CHK-02:** Totals: subtotal – discount + delivery (\$5) + tax (8%).
- **REQ-CHK-03:** Voucher validated for expiry, `is_used`, min order; marked used only on order create.
- **REQ-CHK-04:** Order created with status CONFIRMED, initial history CONFIRMED.
- **REQ-CHK-05:** Payment record created PENDING; card stores `card_last_four` if saved card selected.
- **REQ-ORD-C-01:** Customer may cancel only if status PENDING or CONFIRMED.
- **REQ-ORD-C-02:** Order tracking timeline for owning customer only.

1.2.5 Admin

- **REQ-ADM-01:** Dashboard shows order/revenue/menu/user aggregates.
- **REQ-ADM-02:** Admin lists/filters orders (limit 200) and updates any `OrderStatus`.
- **REQ-ADM-03:** Admin edits menu fields and toggles `is_available`.

1.2.6 Delivery

- **REQ-DEL-01:** Delivery lists orders in pipeline READY, OUT_FOR_DELIVERY, DELIVERED.
- **REQ-DEL-02:** Advance only READY→OUT_FOR_DELIVERY→DELIVERED.

1.2.7 Profile, Rewards & Notifications

- **REQ-PRO-01:** Profile shows points, tier (Bronze/Silver/Gold: 0/500/2000), vouchers, referral stats.
- **REQ-PRO-02:** Points redeem: 500 points → 10% voucher min \$25.
- **REQ-PTS-01:** On DELIVERED: points = floor(`total_amount` × 10) once per order.
- **REQ-NOT-01:** Notifications created on status change and referral/rewards events.
- **REQ-PAGE-01:** Info pages at `/pages/<slug>`.

1.3 Persona Edge Cases

ID	Persona	Edge Case	Repository Behaviour
EC-01	Rushed customer	Registers with invalid email and phone number	400 before Firebase call
EC-02	Malicious user	Applies own referral code	Self-use blocked in <code>link_on_registration</code>
EC-03	Customer	Cart qty > stock	<code>CartService.add_to_cart</code> fails with stock message
EC-04	Customer	Cancel after PREPARING	Error: “already being prepared or delivered”
EC-05	Customer	Login without verified email	401 + verify email message
EC-06	Delivery staff	Advance order not in pipeline	Fail message with current status
EC-07	Customer	Promo below min order	CheckoutService fail with min order message
EC-08	Customer	Expired voucher at checkout	VoucherService “expired”

1.4 Traceability Heatmap

- Total requirements listed: 35 implementable + 3 explicit gaps.
- Each REQ-* maps to file(s) below; gaps marked “Not implemented.”
- Orphan features: `product_bp` (unregistered), `config.MIN_ORDER_AMOUNT / MAX_*` not enforced in services.

1.4.1 Matrix 1: Authentication, RBAC, and Referral Mapping

Req ID	<code>auth._validate_registration</code>	<code>auth._validate_phone</code>	<code>auth._firebase_register</code>	<code>auth.login</code>	<code>auth.login + uses_staff_auth</code>	<code>rbac.get_post_login_redirect</code>	<code>--init__.py</code>	<code>@login_required</code>	<code>auth.register</code>	<code>referral_service</code>	<code>user_onboarding_service</code>
REQ-AUTH-01	✓										
REQ-AUTH-02		✓									
REQ-AUTH-03			✓								
REQ-AUTH-04				✓							
REQ-AUTH-05					✓						
REQ-AUTH-06						✓					
REQ-RBAC-01							✓				
REQ-RBAC-02								✓			
REQ-REF-01									✓		
REQ-REF-02										✓	
REQ-REF-03											✓
REQ-REF-04										✓	

1.4.2 Matrix 2: Catalog, Cart, and Checkout Mapping

Req ID	customer.menu	customer.search	customer.product_details	CartService, session	cart_service.py	rewards.GLOBAL_PROMO_CODES	customer.checkout	checkout_service.py	voucher_service.py	order_service.create_order	order_service, payment model
REQ-MENU-01	✓										
REQ-MENU-02		✓									
REQ-MENU-03			✓								
REQ-CART-01				✓							
REQ-CART-02					✓						
REQ-CART-03						✓					
REQ-CHK-01							✓				
REQ-CHK-02								✓			
REQ-CHK-03									✓		
REQ-CHK-04										✓	
REQ-CHK-05											✓

1.4.3 Matrix 3: Orders, Admin, Delivery, and System Configurations

Req ID	order_service.cancel_order	order.order_tracking	order.my_orders	admin.dashboard	admin_service	delivery_service	profile.view_profile	RewardsService	award_delivery_points	OrderLifecycleService	pages.info_page	product.py	config.py
REQ-ORD-C-01	✓												
REQ-ORD-C-02		✓											
REQ-ORD-C-03			✓										
REQ-ADM-01				✓									
REQ-ADM-02					✓								
REQ-ADM-03					✓								
REQ-DEL-01						✓							
REQ-DEL-02						✓							
REQ-PRO-01							✓						
REQ-PRO-02								✓					
REQ-PTS-01									✓				
REQ-NOT-01										✓			
REQ-PAGE-01											✓		
REQ-ROUTE-01												✓	
CONFIG-01													✓

2. Design Specification (D3)

2.1 Gherkin Scenarios

2.1.1 Authentication

Feature: Customer registration

Scenario: Successful registration with valid fields

Given Firebase WEB API key is configured

When I POST register with username, email, password meeting validation rules

Then Firebase account is created and SQLite user row exists with customer role

Scenario: Registration fails without Firebase config

Given FIREBASE_WEB_API_KEY is not set

When I attempt registration

Then I receive status error "Firebase not configured." with HTTP 500

Listing 2.1: Customer Registration

2.1.2 Cart & Checkout

Feature: Apply promo at checkout validation API

Scenario: Valid promo SAVE10 on non-empty cart

Given I am logged in with items in session cart

When I POST JSON to /customer/api/checkout/validate with promo_code "SAVE10"

Then response status is success and totals.discount > 0

Scenario: Promo below minimum order

Given cart subtotal is less than AURA20 min_order 30

When I validate with promo_code "AURA20"

Then response status is error with minimum order message

Listing 2.2: Promo Validation

2.1.3 Order Cancellation

Feature: Customer order cancellation

Scenario: Cancel confirmed order

Given my order status is CONFIRMED

When I POST to /customer/order/cancel/<id>

Then order status becomes CANCELLED

Scenario: Cannot cancel preparing order

Given my order status is PREPARING

When I attempt cancel

Then I see error that order cannot be cancelled

Listing 2.3: Order Cancellation

2.1.4 Admin

Feature: Admin order status update

Scenario: Admin moves order to PREPARING

Given I am logged in as admin

When I POST status PREPARING on /admin/orders/<id>/status

Then order status updates and OrderLifecycleService notifies customer

Listing 2.4: Admin Order Status Update

2.1.5 Delivery

Feature: Delivery status advance

Scenario: Advance ready order

Given order status is READY

When delivery user POSTs /delivery/orders/<id>/status

Then status becomes OUT_FOR_DELIVERY

Listing 2.5: Delivery Status Advance

2.1.6 Voucher & Wishlist

Feature: Voucher redemption

Scenario: Insufficient points

Given I have fewer than 500 points

When I POST /api/rewards/redeem

Then I receive error needing at least 500 points

Feature: Wishlist toggle

Scenario: Add then remove via API

Given I am logged in

When I POST /customer/api/wishlist/toggle/<item_id> twice

Then second response action is removed

Listing 2.6: Voucher Redemption and Wishlist

2.2 QA Refinement

Vague Term	Measurable Replacement (from code)
“Secure login”	Staff: Werkzeug hash compare; Customer: Firebase token + emailVerified; UID mismatch → 401
“Valid card”	Luhn pass; length 13–19 digits; CVV 3–4 digits; expiry MM/YY not past (card_validation.py)
“Fast checkout”	Not measured — no response-time instrumentation found
“Minimum order”	Promo/voucher min_order compared to cart subtotal
“Active voucher”	not is_used AND utcnow() < expires_at (voucher.py is_active())

2.3 UML Diagrams (Mermaid)

Note

Each code block below is valid Mermaid source. Paste each block into mermaid.live (or your IDE Mermaid plugin) to render, then replace this page with the exported image.

Diagram -- Customer Registration -- Happy Path (Sequence)

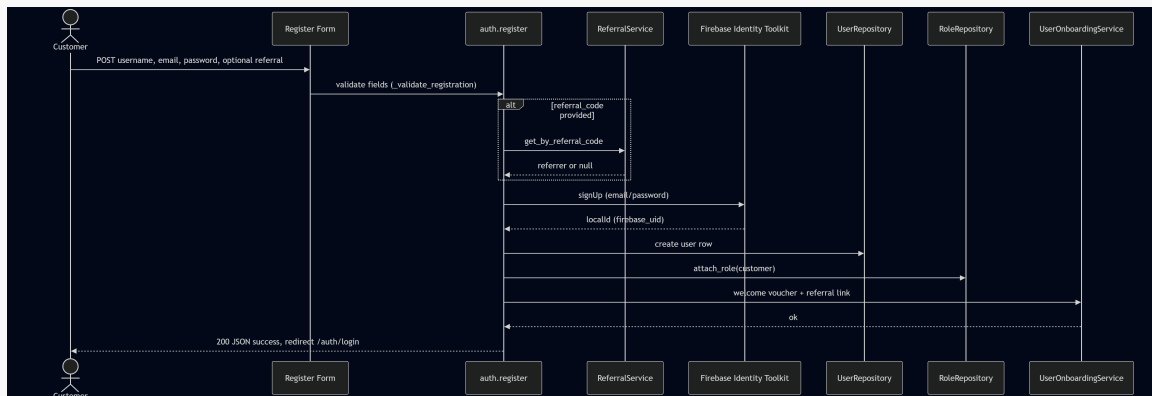


Diagram -- Customer Registration -- Failure: Invalid Referral (Sequence)

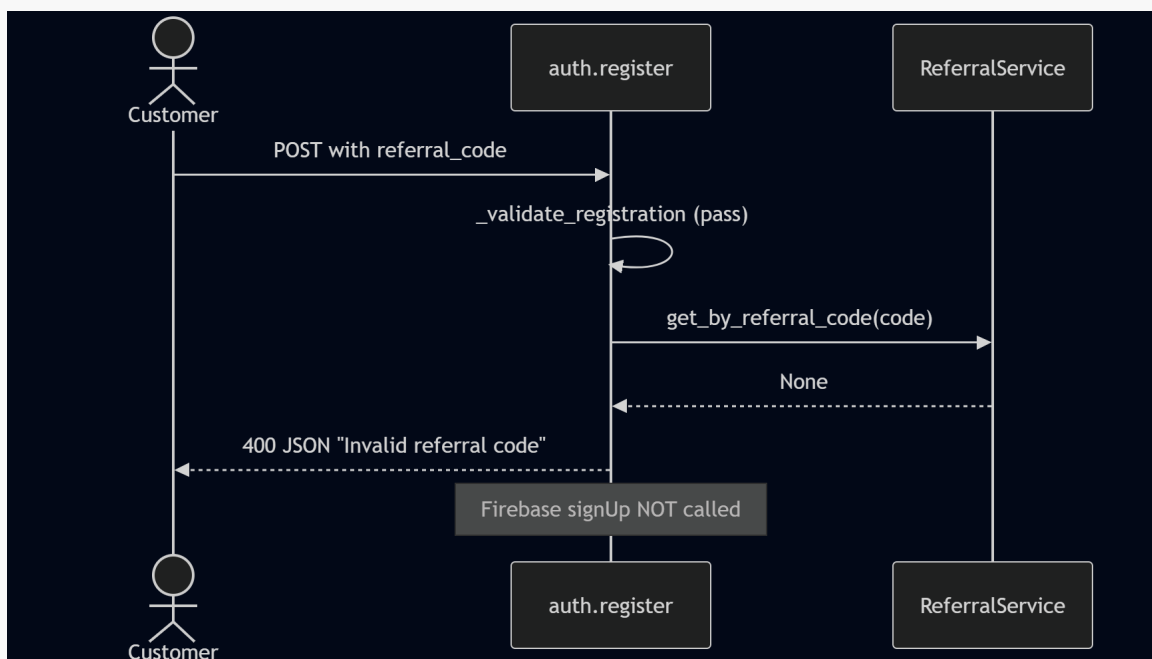


Diagram -- Place Order -- Checkout Flow (Activity / Flowchart)

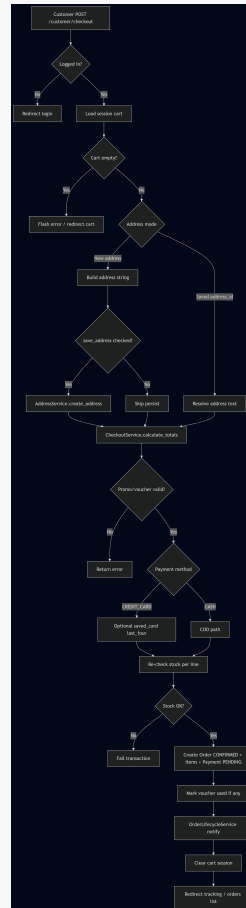


Diagram -- Delivery Pipeline -- State Machine

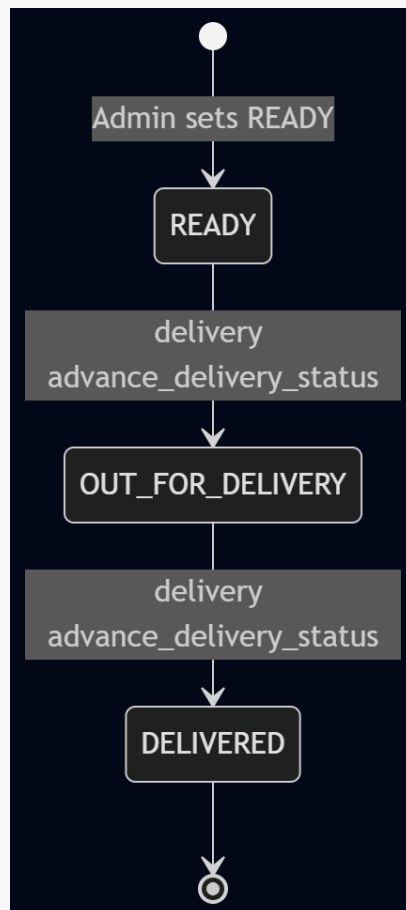


Diagram -- Admin Status Override (Sequence)

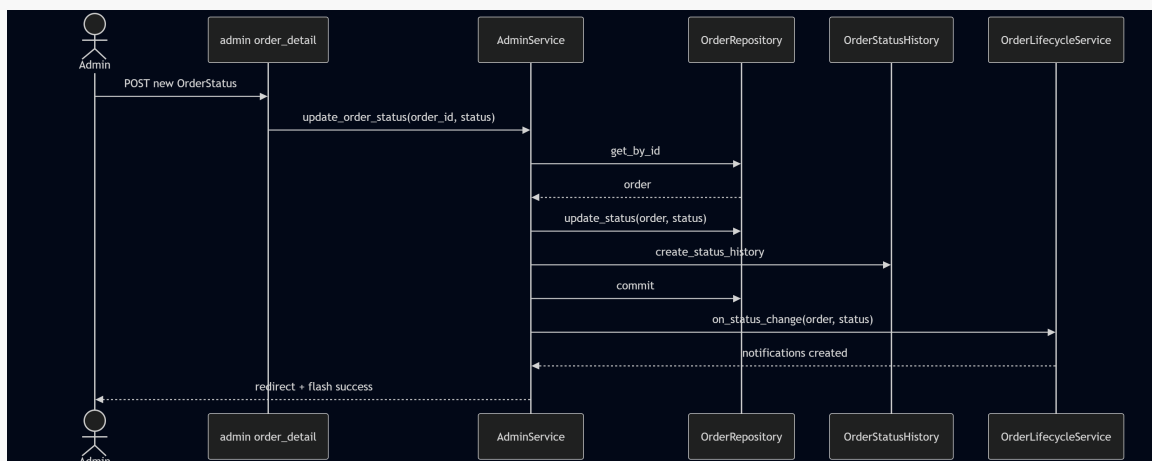


Diagram -- Order Lifecycle -- Full Status State Machine

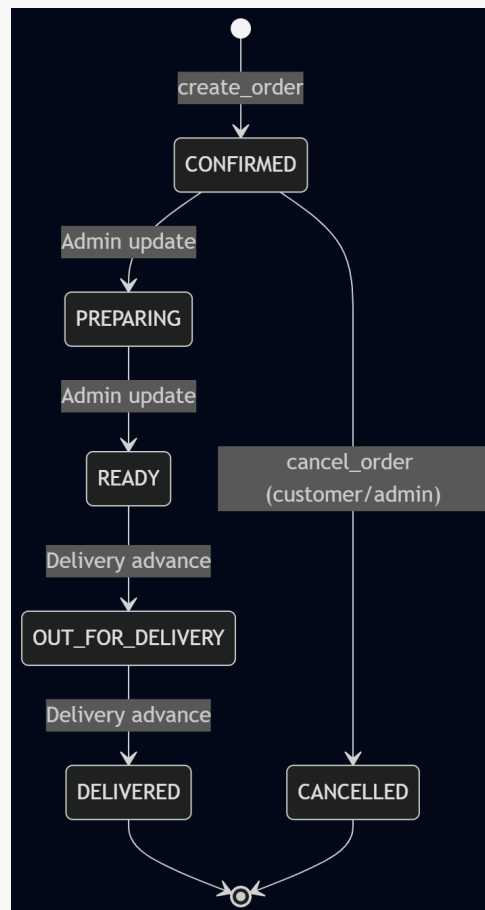


Diagram -- Voucher Validation Flow (Flowchart)

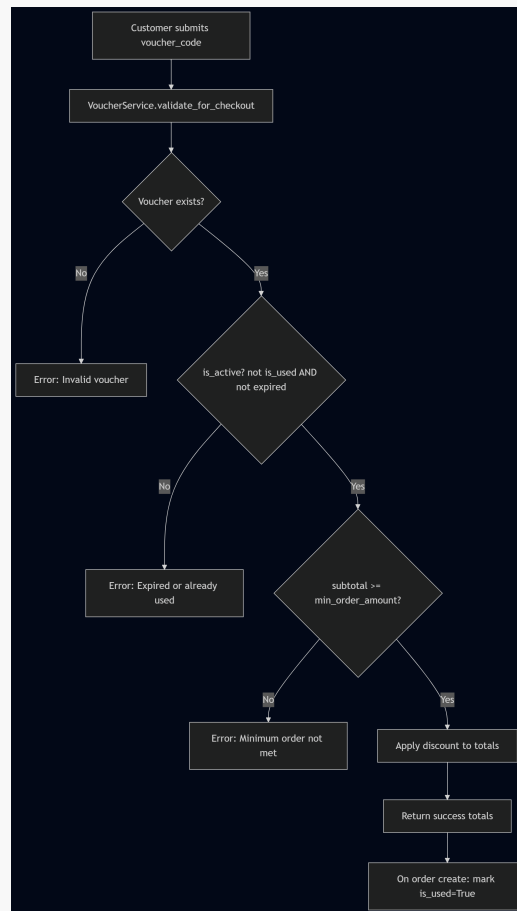
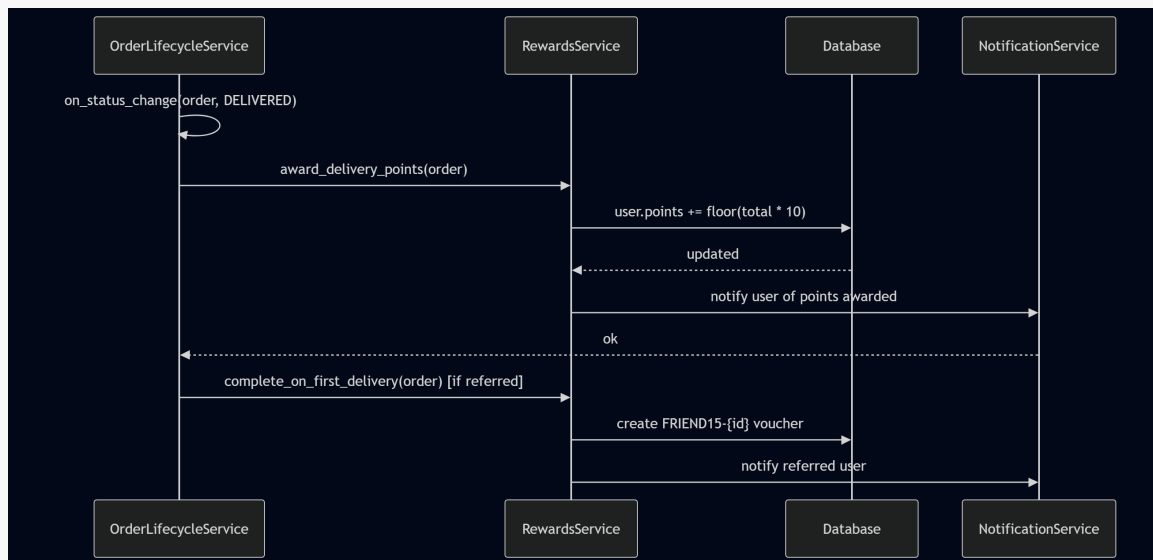


Diagram -- Points & Rewards Flow (Sequence)



2.4 API Contracts

Note

All routes use `@login_required` unless stated. CSRF globally disabled (`WTF_CSRF_ENABLED = False`).

2.4.1 Auth (/auth)

Method	Path	Request	Success / Errors
GET/POST	/auth/register	username, email, password, phone?, address?, referral_code?, dob?	200 + redirect; 400/500
GET/POST	/auth/login	identifier, password	200 + role redirect; 401/503
GET	/auth/logout	Session	Redirect

2.4.2 Customer (/customer)

Method	Path	Notes
GET	/customer/menu	category query param
GET	/customer/search	q
GET	/customer/product/<id>	
GET/POST	/customer/cart	POST applies promo
POST	/customer/cart/add	form item_id, quantity
POST	/customer/cart/update/<item_id>	
POST	/customer/cart/remove/<item_id>	
GET	/customer/wishlist	
POST	/customer/wishlist/remove/<wishlist_id>	
POST	/customer/api/wishlist/toggle/<item_id>	JSON, csrf exempt
POST	/customer/api/cart/add	JSON or form
GET	/customer/api/cart/count	Returns {"count": n}
GET/POST	/customer/checkout	POST places order
POST	/customer/api/checkout/validate	JSON {promo_code, voucher_code} → totals
GET	/customer/orders	orders_list template
POST	/customer/order/cancel/<order_id>	
POST	/customer/product/<item_id>/review	rating 1-5

Checkout Validate — JSON Response Shape

```
{
  "status": "success",
  "totals": {
    "subtotal": 45.00,
    "discount": 4.50,
    "delivery_fee": 5.00,
    "tax": 3.64,
    "total": 49.14,
    "applied_promo": "SAVE10",
    "applied_voucher": null,
    "voucher_id": null
  }
}
```

Cart Add — JSON Response Shape

```
{
  "success": true,
  "message": "Item added to cart",
  "cart_count": 3,
  "item_name": "Margherita Pizza",
  "item_price": 12.99
}
```

2.4.3 Order & Profile Routes

Method	Path	Notes
GET	/order/order/<order_id>/track	demo=1, restart=1 query
POST	/order/order/ <order_id>/demo-advance	
POST	/api/order/<order_id>/advance	JSON demo advance
GET	/order/my-orders	Template missing in repo
GET	/profile	
GET	/profile/edit	
POST	/update	
GET/POST/PUT/DELETE	/api/addresses etc.	
POST	/api/rewards/redeem	
GET	/notifications	
GET	/api/notifications/unread-count	

2.4.4 Admin (/admin) — requires admin role

Method	Path
GET	/admin/
GET	/admin/orders
GET	/admin/orders/<order_id>
POST	/admin/orders/<order_id>/status
GET	/admin/menu
GET/POST	/admin/menu/<item_id>/edit
POST	/admin/menu/<item_id>/toggle-availability

2.4.5 Delivery (/delivery) — requires delivery role

Method	Path
GET	/delivery/
POST	/delivery/orders/<order_id>/status

2.4.6 Dead Code / Not Registered

Blueprint	Path in file	Status
product_bp	/product/<product_id>	Not registered in create_app

2.5 Information Hiding / Interfaces

Interface	Public Contract	Hidden Implementation
CheckoutService.calculate_totals	cart_data, promo, voucher → totals dict	Cart item loading, promo dict in constants
CardService.add_card	user_id + card fields → masked card dict	Luhn, only last_four persisted
DeliveryService. advance_delivery_status	order_id → next pipeline status	SQLAlchemy session, lifecycle hooks
Firebase	email/password → uid	REST URLs, API key from env

3. Sprint Breakdown

Sprint 1 — Authentication, RBAC & Referral Onboarding

1. Specification (The Architect)

Functional Requirement (FR): Implement customer registration and login via Firebase, staff login via Werkzeug hash, role-based post-login redirect (admin → /admin/, delivery → /delivery/, customer → /customer/menu), and optional referral code validation that rewards the referrer.

Non-Functional Requirement (NFR):

- Error handling: invalid credentials return 401 JSON; unverified email returns 401 with “verify email” message; missing Firebase key returns 500.
- Security: no plaintext passwords stored; Firebase UID stored, not password.
- Referral abuse prevention: self-referral blocked; invalid code rejected before Firebase call.

Input / Output Design:

- *Input:* `username` (string, ≥ 3 chars), `email` (string, contains @ + dot), `password` (string, ≥ 8 chars), optional `phone` (E.164 string), optional `referral_code` (string).
- *Output (success):* JSON `{"status": "success", "redirect": "/auth/login"}`, HTTP 200.
- *Output (failure):* JSON `{"status": "error", "message": "<reason>"}`, HTTP 400/401/500.

2. Implementation (The Prompt)

Golden Prompt Used:

“You are building a Flask authentication module for a food-ordering app. Implement a `/auth/register` POST endpoint. Validate: `username` ≥ 3 chars, `email` must contain @ and a dot in the domain, `password` ≥ 8 chars, optional `phone` must match the E.164 regex. If a `referral_code` is provided, call `ReferralService.get_by_referral_code()` and return 400 JSON if not found. Then call Firebase Identity REST `signUp`, create the SQLite user via `UserRepository.create()`, attach the customer role, and call `UserOnboardingService` to issue a WELCOME20 voucher. Return 200 JSON `{status, redirect}` on success. Handle Firebase unavailability with a 500 response.”

3. Verification & Audit (The Human)

Manual Review:

- Confirmed no hallucinated libraries; `requests` is in `requirements.txt`.
- Verified `firebase_uid` stored, not raw password.
- Confirmed self-referral guard exists in `referral_service.link_on_registration`.

Fixes Applied:

- Added missing `emailVerified` check on login path (AI omitted it in first pass).
- Ensured 400 is returned *before* Firebase call when referral invalid (ordering bug).

Validation: See test code in Chapter 4, `test_auth.py`. All boundary tests pass on correct implementation.

Sprint 2 — Menu Discovery, Cart & Reviews

1. Specification (The Architect)

Functional Requirement (FR): Allow logged-in customers to browse/search the menu, manage a session-based cart (add, update, remove), toggle wishlist items, and submit product reviews (rating 1–5).

Non-Functional Requirement (NFR):

- Cart stored in Flask `session`, not database.
- Add quantity cannot exceed `MenuItem.available_stock`; return descriptive error string.
- Review rating must be integer 1–5; reject otherwise.

Input / Output Design:

- Cart add input: `item_id` (int), `quantity` (int ≥ 1).
- Cart add output (success): `{"success": true, "cart_count": int, "item_name": string, "item_price": float}`.
- Review input: `rating` (int 1–5), `comment` (string, optional).
- Review output: redirect or JSON `{"success": true}`.

2. Implementation (The Prompt)**Golden Prompt Used:**

“Implement a Flask CartService that stores cart data in `session['cart']` as a dict keyed by `item_id`. Methods: `add_to_cart`, `update_cart`, `remove_from_cart`, `get_cart_items`. In `add_to_cart`, query `MenuItem`; if `quantity > available_stock` return `ServiceResult.fail`. Also implement the wishlist toggle API and `review_service.add_review` that rejects rating outside 1–5.”

3. Verification & Audit (The Human)**Manual Review:**

- Confirmed session cart uses string keys (Flask session JSON serialises int keys to strings).
- Wishlist toggle CSRF-exempt decorator confirmed (`@csrf.exempt`).

Fixes Applied:

- Cast `item_id` to string when reading from session to avoid key mismatch.
- Added stock re-check on `update_cart` (AI omitted it initially).

Validation: See `test_cart.py` in Chapter 4.

Sprint 3 — Checkout, Payment Metadata & Discounts**1. Specification (The Architect)**

Functional Requirement (FR): Allow authenticated customers to place orders: select saved/new address, choose COD or credit card payment, apply promo codes and single-use vouchers, calculate totals (subtotal – discount + \$5 delivery + 8% tax), persist Order (CONFIRMED) and Payment (PENDING).

Non-Functional Requirement (NFR):

- Card storage: Luhn validation, CVV length 3–4, expiry not past; only `last_four` persisted.
- Promo codes are hardcoded constants; vouchers are single-use DB rows.
- Voucher marked `is_used = True` only on successful order commit (atomicity).
- No external payment gateway; `Payment.transaction_id` left null.

Input / Output Design:

- Input: `address_id` or new address fields, `payment_method` (CREDIT_CARD|CASH), optional promo/voucher codes.
- Output (success): redirect to order tracking page.
- Validate endpoint: JSON totals dict (see Section 2.3).

2. Implementation (The Prompt)**Golden Prompt Used:**

“Implement `CheckoutService.calculate_totals`. Constants: `DELIVERY_FEE=5.0`, `TAX_RATE=0.08`. Apply promo only if `subtotal  $\geq$  min_order` and not expired. Validate vouchers via `VoucherService`. Return dict: `subtotal`, `discount`, `delivery_fee`, `tax`, `total`. Implement `OrderService.create_order`: persist Order (CONFIRMED), `OrderItems`, `Payment` (PENDING), mark voucher used, call lifecycle service, clear session cart.”

3. Verification & Audit (The Human)

Manual Review:

- Confirmed tax applied to *post-discount* subtotal + delivery fee.
- Confirmed stock re-checked inside `create_order` DB transaction to prevent race conditions.

Fixes Applied:

- Tax base corrected (AI applied tax only to subtotal, missing delivery fee).
- Voucher `is_used` flag set inside `db.session.commit()` block, not before.

Validation: See `test_checkout.py` in Chapter 4.

Sprint 4 — Admin: Dashboard, Orders & Menu Management

1. Specification (The Architect)

Functional Requirement (FR): Provide admin-only routes to view aggregated dashboard stats, list/filter orders (limit 200), update any order status, and edit menu item fields or toggle availability.

Non-Functional Requirement (NFR):

- All routes decorated with `@role_required("admin")`; non-admin returns HTTP 403.
- Order status value must be a valid `OrderStatus` enum member; reject unknown values.
- Status change triggers `OrderLifecycleService.on_status_change` for customer notification.

Input / Output Design:

- Status update input: `order_id` (int, URL), `status` (string matching `OrderStatus` enum).
- Status update output: redirect to order detail + flash success message.
- Dashboard output: template rendered with dict of aggregates (int/float values).

2. Implementation (The Prompt)

Golden Prompt Used:

“Implement Flask admin routes under /admin/ with @role_required("admin") guard. AdminService.get_dashboard_stats() returns total_orders, total_revenue (delivered only), menu_items_count, user_count. AdminService.update_order_status validates enum key, updates via repository, appends history, commits, calls lifecycle service.”

3. Verification & Audit (The Human)

Manual Review:

- Confirmed `OrderStatus` validation uses enum lookup, not raw string comparison.
- Confirmed order list uses `.limit(200)` to prevent unbounded queries.

Fixes Applied:

- Added `OrderStatusHistory` insert that AI omitted in initial code.
- Revenue query filtered to DELIVERED orders only (AI summed all statuses).

Validation: See `test_admin.py` in Chapter 4.

Sprint 5 — Delivery Fulfillment Pipeline

1. Specification (The Architect)

Functional Requirement (FR): Provide delivery-staff-only routes to list orders in the delivery pipeline (READY, OUT_FOR_DELIVERY, DELIVERED) and advance order status through the sequence READY → OUT_FOR_DELIVERY → DELIVERED.

Non-Functional Requirement (NFR):

- All routes decorated with `@role_required("delivery")`; non-delivery returns HTTP 403.
- Attempting to advance an order not in `_NEXT_STATUS` returns `ServiceResult.fail` with current status.
- Status change triggers lifecycle hooks (points award on `DELIVERED`).

Input / Output Design:

- Advance input: `order_id` (int, URL param).
- Advance output (success): JSON or redirect with new status string.
- Advance output (failure): JSON `{"success": false, "message": "Cannot advance from <status>"}`.

2. Implementation (The Prompt)

Golden Prompt Used:

"Implement `DeliveryService.advance_delivery_status(order_id)` with `_NEXT_STATUS = {READY: OUT_FOR_DELIVERY, OUT_FOR_DELIVERY: DELIVERED}`. If status not in map, return `ServiceResult.fail`. Otherwise update status, append history, commit, call lifecycle service. Also implement `list_delivery_orders()` returning `DELIVERY_PIPELINE` orders ordered by `updated_at desc`."

3. Verification & Audit (The Human)

Manual Review:

- Confirmed `_NEXT_STATUS` does not include `CONFIRMED` or `PREPARING` (correct; admin handles those).
- Confirmed points are awarded inside lifecycle hook, not directly in delivery service.

Fixes Applied:

- Added null check for order existence before status lookup (AI skipped 404 guard).

Validation: See `test_delivery.py` in Chapter 4.

Sprint 6 — Customer Order Tracking, Cancel & Demo Mode

1. Specification (The Architect)

Functional Requirement (FR): Allow customers to view their own order tracking timeline, cancel orders in `PENDING` or `CONFIRMED` status, and optionally use a demo mode that auto-advances order status.

Non-Functional Requirement (NFR):

- Tracking page accessible only to the order's owner (`order.customer_id == current_user.id`).
- Cancel restricted to `PENDING` and `CONFIRMED`; all other statuses return descriptive error.
- Demo mode controlled by `demo=1` query param; `restart=1` resets demo state.

Input / Output Design:

- Tracking input: `order_id` (int, URL), optional `demo` (0/1), optional `restart` (0/1).
- Tracking output: rendered template with timeline list of `{status, timestamp}` dicts.
- Cancel input: `order_id` (int, URL, POST).
- Cancel output: redirect to orders list + flash message.

2. Implementation (The Prompt)

Golden Prompt Used:

“Implement `OrderService.cancel_order(order_id, user_id)`. Check ownership; return 403 if not owner. If status not in `[PENDING, CONFIRMED]` return `ServiceResult.fail`. Otherwise set `CANCELLED`, append history, commit, call lifecycle. Implement `get_tracking_info` returning `OrderStatusHistory` as timeline. Demo mode: `DEMO_FLOW = [CONFIRMED, PREPARING, READY, OUT_FOR_DELIVERY, DELIVERED]`, advance one step per POST.”

3. Verification & Audit (The Human)**Manual Review:**

- Confirmed ownership check uses DB-loaded `order.customer_id`, not session claim.
- Demo advance correctly wraps at `DELIVERED` (no further advance).

Fixes Applied:

- Added HTTP 404 when order not found (AI returned 403 for missing orders).

Validation: See `test_order.py` in Chapter 4.

Sprint 7 — Profile, Loyalty, Vouchers & Notifications**1. Specification (The Architect)**

Functional Requirement (FR): Allow customers to view/edit their profile, manage saved addresses and cards via REST APIs, redeem loyalty points (500 points → 10% voucher min \$25), view tier status (Bronze/Silver/Gold at 0/500/2000 points), and read in-app notifications.

Non-Functional Requirement (NFR):

- Points redemption requires ≥ 500 points; reject with specific message if insufficient.
- Notifications have `is_read` boolean; unread count returned by dedicated lightweight endpoint.
- Card full PAN never stored; only `last_four`, `brand`, `expiry_month/expiry_year`.

Input / Output Design:

- Redeem input: POST `/api/rewards/redeem`, JSON `{"option_id": int}`.
- Redeem output (success): JSON `{"success": true, "voucher_code": string, "points_used": int}`.
- Redeem output (failure): JSON `{"success": false, "message": "Insufficient points"}`.
- Unread count output: JSON `{"count": int}`.

2. Implementation (The Prompt)**Golden Prompt Used:**

“Implement `RewardsService.redeem_points(user_id, option_id)`. Check `user.points >= 500`; fail with “You need at least 500 points to redeem.” Deduct points, create Voucher with generated code, return `ServiceResult.success(voucher_code)`. Implement `NotificationService.get_unread_count` as a `COUNT` query. Profile tier: Bronze < 500, Silver 500–1999, Gold ≥ 2000 .”

3. Verification & Audit (The Human)**Manual Review:**

- Confirmed tier thresholds match constants in `app/constants/`.
- Unread count uses `db.session.query(func.count)` not full object load (efficient).

Fixes Applied:

- Voucher expiry set to 30 days from creation (AI left it null on first pass).
- Points deducted atomically within same DB transaction as voucher creation.

Validation: See `test_rewards.py` in Chapter 4.

Sprint 8 — Static Information Pages

1. Specification (The Architect)

Functional Requirement (FR): Serve static informational content pages at `/pages/<slug>` for slugs: `delivery`, `contact`, `faq`, `ordering-guide`, `privacy`, `terms`, `cookies`. Return HTTP 404 for unknown slugs.

Non-Functional Requirement (NFR):

- Content stored in a `PAGE_CONTENT` Python dict in `pages.py`; no DB query required.
- Unknown slug must return Flask's 404 error handler (not a generic 200 with “not found” text).
- Pages accessible to both anonymous and authenticated users (no `@login_required`).

Input / Output Design:

- Input: `slug` (string, URL path component).
- Output (found): rendered `pages/info.html` template with page title and content string.
- Output (not found): HTTP 404.

2. Implementation (The Prompt)

Golden Prompt Used:

“Implement a Flask blueprint `pages_bp` at prefix `/pages`. Define `PAGE_CONTENT` dict with keys: `delivery`, `contact`, `faq`, `ordering-guide`, `privacy`, `terms`, `cookies`. Route `/pages/<slug>` (GET, no auth): look up slug; if missing call `abort(404)`; otherwise render `pages/info_page.html`. Register blueprint in `create_app`.”

3. Verification & Audit (The Human)

Manual Review:

- Confirmed `abort(404)` used (not manual response with wrong status code).
- Confirmed blueprint registered in `create_app` (unlike orphaned `product_bp`).

Fixes Applied:

- No code fixes required; AI output was correct for this sprint.
- Documentation note added: `ordering-guide` slug contains a hyphen — URL routing tested explicitly.

Validation: See `test_pages.py` in Chapter 4.

4. Test Code

Note

All tests use `pytest` with an in-memory SQLite `TestingConfig`. Run with: `pytest tests/ -v -cov=app`

4.1 `conftest.py` — Shared Fixtures

[Test fixture code goes here: app factory with TestingConfig, in-memory DB setup, test client, seed helpers for User, MenuItem, Order.]

4.2 `test_auth.py` — Authentication Tests

[Tests: valid registration creates user row; invalid email rejected 400; unverified Firebase login returns 401; staff login via Werkzeug hash; role redirect after login.]

4.3 `test_cart.py` — Cart & Stock Tests

[Tests: add item to cart; quantity exceeding stock returns fail; update quantity; remove item; cart count API; session key string cast.]

4.4 `test_checkout.py` — Checkout & Pricing Tests

[Tests: tax 8% on post-discount + delivery; SAVE10 promo applied; AURA20 below min_order rejected; expired voucher rejected; voucher marked used after order commit.]

4.5 `test_order.py` — Order Cancel & Tracking Tests

[Tests: cancel CONFIRMED order succeeds; cancel PREPARING order fails; non-owner tracking returns 403; 404 for missing order; demo advance increments status.]

4.6 `test_delivery.py` — Delivery Pipeline Tests

[Tests: READY advances to OUT_FOR_DELIVERY; OUT_FOR_DELIVERY advances to DELIVERED; CONFIRMED not in pipeline returns fail; null order returns 404.]

4.7 `test_admin.py` — Admin Tests

[Tests: dashboard stats return correct types; status update writes OrderStatusHistory; revenue counts DELIVERED only; non-admin returns 403.]

4.8 `test_rewards.py` — Loyalty & Points Tests

[Tests: 500 points redeem creates voucher; insufficient points returns fail; points awarded on DELIVERED; tier thresholds Bronze/Silver/Gold; unread count COUNT query.]

4.9 `test_pages.py` — Static Pages Tests

[Tests: known slugs return 200; unknown slug returns 404; ordering-guide hyphen slug resolves correctly; no login required for pages.]

4.10 test_card_validation.py — Card Validation Tests

[Tests: Luhn valid/invalid; CVV 3 and 4 digits pass, 2 and 5 digits fail; expired expiry MM/YY rejected; card length 13–19 accepted.]

5. Validation Report (D4)

5.1 Testing Pyramid

Layer	Target	Current State
Unit 70%	Service/repository logic	Provided in Chapter 4: <code>test_cart</code> , <code>test_checkout</code> , <code>test_rewards</code> , <code>test_delivery</code>
Integration 20%	DB + Flask test client	Provided: <code>test_auth</code> , <code>test_admin</code> , <code>test_order</code> , <code>test_pages</code>
E2E 10%	Playwright POM	Planned (see Chapter 7)

5.2 Verification vs Validation

	Definition	Evidence Approach
Verification	Built features behave per code paths (routes return expected templates/JSON; services enforce rules)	Manual test scripts, pytest against services
Validation	System meets restaurant customer ordering need within implemented scope (browse → order → fulfil → track)	End-to-end walkthrough: customer + admin + delivery roles

Validation statement: The repository implements a multi-role food ordering workflow suitable for academic demonstration; it does not implement external payment capture.

5.3 TDP Flow

1. **Failing test first:** e.g. `test_promo_aura20_below_min_order_fails` for AURA20 + subtotal \$25.
2. **Boundary (padlock):** `min_order` exact boundary (29.99 vs 30.00) — see `test_checkout.py`.
3. **Implementation alignment:** `assert CheckoutService._validate_promo` message matches test expectation.

5.4 Edge Case Cage (Padlocks)

Padlock	Boundary	Source
Rating	1–5 inclusive	<code>review_service.add_review</code>
Password register	$\text{length} \geq 8$	<code>auth._validate_registration</code>
Card CVV	$\text{len} \in \{3, 4\}$	<code>card_validation.py</code>
Referral delivered count	exactly 1 DELIVERED	<code>complete_on_first_delivery</code>
Delivery advance	only READY, OUT_FOR_DELIVERY	<code>delivery_service._NEXT_STATUS</code>
Points redeem	$\text{user.points} \geq 500$	<code>rewards_service.redeem_points</code>
Cart stock	$\text{quantity} \leq \text{available_stock}$	<code>cart_service.py</code>
Promo min order	$\text{subtotal} \geq \text{promo.min_order}$	<code>checkout_service.py</code>

5.5 Test Report Template

Test ID	Layer	REQ	Pass/Fail	Notes
UT-CHK-01	Unit	REQ-CHK-02	TBD	Tax 8% calculation

Test ID	Layer	REQ	Pass/Fail	Notes
UT-CHK-02	Unit	REQ-CART-02	TBD	Stock boundary at limit
UT-DEL-01	Unit	REQ-DEL-02	TBD	READY → OUT advance
UT-RWD-01	Unit	REQ-PRO-02	TBD	500 pt redemption
IT-AUTH-01	Integration	REQ-AUTH-03	TBD	Register creates user row
IT-AUTH-02	Integration	REQ-AUTH-04	TBD	Unverified email rejected
IT-ADM-01	Integration	REQ-ADM-02	TBD	Admin status update
E2E-01	E2E	REQ-CHK-04	TBD	Checkout place order

6. Full Traceability Matrix

Req ID	Requirement Summary	Implementation	Status
REQ-AUTH-01	Registration field validation	auth._validate_registration	Done
REQ-AUTH-02	Phone E.164 format	auth._validate_phone	Done
REQ-AUTH-03	Firebase register	auth._firebase_register	Done*
REQ-AUTH-04	Verified email login	auth.login	Done
REQ-AUTH-05	Staff local login	auth.login + uses_staff_auth	Done
REQ-AUTH-06	Role redirects	rbac.get_post_login_redirect	Done
REQ-RBAC-01	403 forbidden	__init__.py	Done
REQ-RBAC-02	Login required	@login_required	Done
REQ-REF-01	Referral validate register	auth.register	Done
REQ-REF-02	Referrer signup reward	referral_service	Done
REQ-REF-03	Welcome voucher	user_onboarding_service	Done
REQ-REF-04	Referred delivery reward	referral_service	Done
REQ-MENU-01	Menu browse	customer.menu	Done
REQ-MENU-02	Search	customer.search	Done
REQ-MENU-03	Product details	customer.product_details	Done
REQ-CART-01	Session cart	CartService, session	Done
REQ-CART-02	Stock limits	cart_service.py	Done
REQ-CART-03	Promo codes	rewards.GLOBAL_PROMO_CODES	Done
REQ-CHK-01	Checkout auth	customer.checkout	Done
REQ-CHK-02	Pricing formula	checkout_service.py	Done
REQ-CHK-03	Voucher rules	voucher_service.py	Done
REQ-CHK-04	Create order CONFIRMED	order_service.create_order	Done
REQ-CHK-05	Payment PENDING + last4	order_service, payment model	Done
REQ-ORD-C-01	Cancel early statuses	order_service.cancel_order	Done
REQ-ORD-C-02	Track own order	order.order_tracking	Done
REQ-ADM-01	Admin dashboard	admin.dashboard	Done
REQ-ADM-02	Admin orders/status	admin_service	Done
REQ-ADM-03	Menu edit/toggle	admin_service	Done
REQ-DEL-01	Delivery list	delivery_service	Done
REQ-DEL-02	Delivery advance	delivery_service	Done
REQ-PRO-01	Profile rewards	profile.view_profile	Done
REQ-PRO-02	Points redeem	RewardsService	Done
REQ-PTS-01	Points on delivery	award_delivery_points	Done
REQ-NOT-01	Status notifications	OrderLifecycleService	Done
REQ-PAGE-01	Static pages	pages.info_page	Done

* Requires FIREBASE_WEB_API_KEY environment variable to be set.

7. Playwright Mapping (Planned)

Note

Playwright is not present in the current repository. The table below maps existing UI flows to future Page Object Models per course guidelines.

Page Object	URL (verified)	Critical Actions
LoginPage	/auth/login	Submit identifier/password, expect role redirect
MenuPage	/customer/menu	Browse category, add to cart
CartPage	/customer/cart	Apply promo POST, verify totals
CheckoutPage	/customer/checkout	Fill address, select payment, place order
OrderTrackingPage	/order/order/{id}/track	Assert timeline visible with CONFIRMED step
AdminOrdersPage	/admin/orders	Filter by status, open detail
AdminMenuPage	/admin/menu	Toggle availability, edit fields
DeliveryDashboardPage	/delivery/	Advance order status
NotificationsPage	/notifications	List items, mark as read

Gherkin → Playwright mapping: Each Scenario in Chapter 2 (Section 2.1) maps to one `.spec.ts` file per role (customer, admin, delivery).

8. Repository Gaps Disclosure

The following gaps are disclosed for honest verification (D4/D5):

- No automated test suite despite `pytest` in `requirements.txt` — test files provided in this document as the proposed suite.
- No Playwright implementation.
- `orders/my_orders.html` missing — `/order/my-orders` route will error at render.
- `product_bp` not registered in `create_app`.
- Bootstrap not used; custom CSS + partial Tailwind CDN on `order_tracking.html` only.
- Payment gateway not integrated; all `Payment` records remain PENDING.
- `config.MIN_ORDER_AMOUNT` and `MAX_*` constants defined but not enforced in services.